

COMPANION
to

ChatGPT Teaches TikZ

Chapter 19: Building a Style System (A)

Contents

19.1	What This Companion Adds	2
19.2	A Style-System Vocabulary	3
19.3	Designing the Vocabulary	4
19.4	Building Styles from Styles	4
19.5	Combining Orthogonal Styles	5
19.6	Appending and Prefixing Options	5
19.7	Broad Defaults with every	6
19.8	Local Exceptions with a Scope	6
19.9	Controlling Where the System Lives	7
19.10	Worked Project: One System, Two Graphs	7
19.11	Debugging Workshop	9
19.12	Exercises	9
19.13	Solutions	10
19.14	ChatGPT Workshop	12
19.15	Final Checklist	12

19.1 What This Companion Adds

Chapter 19 moved from isolated styles to a coherent style system. This Companion develops the architecture: define a small semantic vocabulary, build specialized styles from base styles, use broad defaults cautiously, and keep exceptions local.

Design

A style system is an interface between mathematical structure and visual design. It should be small enough to understand and broad enough to reuse.

19.2 A Style-System Vocabulary

Form	Purpose
<code>\tikzset{...}</code>	Defines a family of project styles.
<code>child/.style={parent,...}</code>	Builds a specialized style from a base style.
<code>/.append style=</code>	Adds options after an existing style definition.
<code>/.prefix style=</code>	Adds options before an existing style definition.
<code>every node/.style=</code>	Establishes defaults for all nodes in scope.
<code>every path/.style=</code>	Establishes defaults for all paths in scope.
<code>every label/.style=</code>	Establishes defaults for labels.
<code>scope</code>	Limits temporary changes and exceptions.

19.3 Designing the Vocabulary

Begin with roles that recur across figures:

```
\tikzset{
  vertex/.style={draw,circle,minimum size=8mm},
  edge/.style={thick},
  construction/.style={densely dashed,gray},
  annotation/.style={font=\small,fill=white}
}
```

Do not define a style merely because one option list occurs once. The system should express the project's recurring visual grammar.

19.4 Building Styles from Styles

A specialized style should contain its base role.

```
\tikzset{
  vertex/.style={draw,circle,minimum size=8mm},
  important vertex/.style={
    vertex,very thick,fill=yellow!20
  }
}
```

If the base vertex size changes, the important vertex inherits the change.

Common Mistake

Do not copy the complete base definition into every specialization. Copies drift apart when the design is revised.

19.5 Combining Orthogonal Styles

Inheritance is useful when one role is a specialized form of another. Separate roles may instead be combined at the point of use.

```
\tikzset{
  vertex/.style={draw,circle,minimum size=8mm},
  selected/.style={very thick,fill=gray!20}
}
\node[vertex,selected] at (0,0) {$A$};
```

This avoids creating many combinations such as selected input vertex, selected output vertex, and selected ordinary vertex.

19.6 Appending and Prefixing Options

An existing style may be extended without rewriting it.

```
\tikzset{
  vertex/.style={draw,circle,minimum size=8mm},
  vertex/.append style={inner sep=1pt}
}
```

Use `/.prefix style` when the new options must be inserted before the existing definition.

```
\tikzset{vertex/.prefix style={font=\small}}
```

When options conflict, their order can determine the result. Keep extensions short and document why their position matters.

19.7 Broad Defaults with every ...

```
\begin{tikzpicture}[
  every node/.style={font=\small},
  every path/.style={line cap=round}
]
...
\end{tikzpicture}
```

These defaults are appropriate because they genuinely apply to every object of the named kind.

Design

Use a broad default only for a broad rule. If a picture contains several kinds of nodes, do not make **every node** a circular vertex style.

19.8 Local Exceptions with a Scope

```
\begin{tikzpicture}[
  vertex/.style={draw,circle,minimum size=8mm}
]
\node[vertex] at (0,0) {$A$};
\begin{scope}[
  vertex/.append style={very thick,fill=gray!20}
]
  \node[vertex] at (2,0) {$B$};
  \node[vertex] at (4,0) {$C$};
\end{scope}
\node[vertex] at (6,0) {$D$};
\end{tikzpicture}
```

Only B and C receive the temporary extension.

19.9 Controlling Where the System Lives

For one figure, place the system in its option list. For a chapter, define it before the chapter's figures. For an entire book, place stable definitions in a style file. A wider scope increases reuse but also increases the consequences of an accidental change.

19.10 Worked Project: One System, Two Graphs

19.10.1 Stage 1: Define the Shared System

```
\tikzset{
  vertex/.style={draw,circle,minimum size=7mm},
  special/.style={vertex,very thick,fill=gray!20},
  edge/.style={thick}
}
```

19.10.2 Stage 2: Use It in Different Geometries

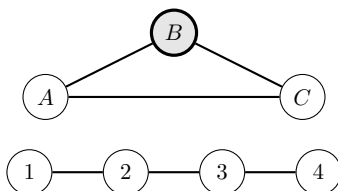
```
\node[vertex] (A) at (0,0) {$A$};
\node[special] (B) at (2,1) {$B$};
\node[vertex] (C) at (4,0) {$C$};
\draw[edge] (A)--(B)--(C)--(A);
```

19.10.3 The Complete Pair

```
\begin{tikzpicture}
  \node[vertex] (A) at (0,0) {$A$};
  \node[special] (B) at (2,1) {$B$};
  \node[vertex] (C) at (4,0) {$C$};
  \draw[edge] (A)--(B)--(C)--(A);
\end{tikzpicture}

\begin{tikzpicture}
  \foreach \i/\x in {1/0,2/1.5,3/3,4/4.5}
    \node[vertex] (v\i) at (\x,0) {$\i$};
  \draw[edge] (v1)--(v2)--(v3)--(v4);
\end{tikzpicture}
```

See



Design

The figures have different structures but share one visual vocabulary. A single system edit updates both.

19.11 Debugging Workshop

19.11.1 A Specialized Style Stops Following Its Parent

Ensure that the child style includes the parent style by name rather than a copied option list.

19.11.2 A Default Affects Annotations

Narrow an overly broad `every node` rule and move vertex-specific options into `vertex`.

19.11.3 A Temporary Change Leaks

Place the append or prefix operation inside a scope and check the scope ending.

19.11.4 The System Has Too Many Names

Combine independent styles and remove names created for one isolated object.

19.12 Exercises

1. Define a four-style vocabulary for an earlier diagram.
2. Build an important-node style from a base node style.
3. Combine a base role and a selected role on one node.

4. Append a common font to an existing node style.
5. Set one genuinely universal **every node** default.
6. Apply a temporary style extension inside a scope.
7. Use one system in two geometrically different figures.
8. Move a stable style system into the shared style file.

19.13 Solutions

19.13.1 Exercises 1–4

```
\tikzset{
  object/.style={draw,minimum size=8mm},
  important/.style={object,very thick},
  selected/.style={fill=yellow!20},
  relation/.style={thick}
}
```

```
\node[object,selected] at (0,0) {$A$};
\tikzset{object/.append style={font=\small}}
```

19.13.2 Exercises 5–7

```
\begin{scope}[object/.append style={fill=gray!15}]  
  ...  
\end{scope}
```

Exercise 7 keeps the style definitions unchanged while the coordinates and paths differ. For Exercise 8, move only stable project-wide definitions.

19.14 ChatGPT Workshop

Ask ChatGPT to evaluate the architecture, not merely shorten the source.

1. Organize these isolated styles into a small semantic hierarchy.
2. Find copied parent definitions that should use style inheritance.
3. Identify broad **every** ... rules that affect unintended objects.
4. Localize this temporary redesign with a scope.
5. Reduce the number of combination styles without changing the figures.

19.15 Final Checklist

Before continuing to Chapter 20, check that you can

1. define a coherent semantic vocabulary;
2. build specialized styles from base styles;
3. combine independent roles at the point of use;
4. append or prefix a short extension;
5. use broad defaults cautiously;
6. localize exceptions with a scope; and
7. reuse one system across several figures.